

1. Component and format map (Apache POI 5.5.1, Java 21)

Artifact	Brings	Formats
poi	POIFS, HSSF, HPSF, usermodel API	.xls, OLE2
poi-ooxml	OPC, XSSF, SXSSF, XWPF, XSLF + poi-ooxml-lite	.xlsx .docx .pptx
poi-ooxml-fu	all schemas; swap for lite on NoClassDefFoundError	OOXML
poi-scratchpad	HWPF, HSLF, HSMF, HDGF, HPBF	.doc .ppt .msg

OOXML = ZIP of XML parts (OPC); OLE2 = POIFS binary container. Coordinates: org.apache.poi:poi-ooxml:5.5.1 (add poi-scratchpad for .doc). Add log4j-api bridge for logging.

2. Which API?

Need	API	Notes
Read/write, random access, small-medium	usermodel XSSF / HSSF	whole file in heap
Write huge .xlsx	SXSSF	write-only, row window
Read huge .xlsx	SAX / event: XSSFWorkbook + SheetContentsHandler	forward, read-only; HSSF EventUserModel for .xls
Read .xlsb	XSSFWorkbook (5.5)	read-only

3. Workbook skeleton

```
try (Workbook wb = new XSSFWorkbook();
    OutputStream out = Files.newOutputStream(path)) {
    Sheet s = wb.createSheet("Books");
    Row r = s.createRow(0);
    r.createCell(0).setCellValue("Title");
    wb.write(out);
} // read: try (Workbook wb = WorkbookFactory.create(in))
```

4. Cells, CellType, DataFormatter, DateUtil

```
switch (cell.getCellType()) { // CellType enum
    case STRING -> cell.getStringCellValue();
    case NUMERIC -> DateUtil.isCellDateFormatted(cell)
        ? cell.getLocalDateTimeCellValue()
        : cell.getNumericCellValue();
    case BOOLEAN, FORMULA, BLANK, ERROR, _NONE -> ...
}
DataFormatter f = new DataFormatter();
String text = f.formatCellValue(cell, evaluator);
```

Formula cell: getCacheFormulaResultType(). Never getCellTypeEnum() (deprecated in 4.x, removed in 5.0). Dates are numbers + a date format; setCellValue(LocalDate) plus a date style.

5. Styles, fonts, formats

```
CellStyle st = wb.createCellStyle(); // once
Font fnt = wb.createFont(); fnt.setBold(true);
st.setFont(fnt);
st.setDataFormat(wb.createDataFormat().getFormat("#,##0.00"));
st.setBorderBottom(BorderStyle.THIN);
cell.setCellStyle(st); // reuse
```

Reuse rule: create each style/font once and share it (cache by key). Per-cell createCellStyle() hits 64,000 styles (.xlsx) or 4,000 (.xls).

6. Formulas and FormulaEvaluator

```
cell.setCellFormula("SUM(C2:C10)");
FormulaEvaluator ev =
    wb.getCreationHelper().createFormulaEvaluator();
ev.evaluateAll(); // or evaluateFormulaCell(cell)
wb.setForceFormulaRecalculation(true); // Excel recalcs
```

Cached values are not computed on write; check that every function is supported by the evaluator. Never setCellFormula with user data.

7. Layout and printing

```
s.createFreezePane(0, 1); s.autoSizeColumn(0);
s.setColumnWidth(1, 20 * 256);
s.setAutoFilter(CellRangeAddress.valueOf("A1:F1"));
s.addMergedRegion(new CellRangeAddress(0, 0, 0, 3));
s.setFitToPage(true); s.setPrintSetup().setLandscape(true);
s.setRepeatingRows(CellRangeAddress.valueOf("1:1"));
wb.setPrintArea(0, "A1:F40");
```

8. Validation, conditional formats, tables, pivots, charts

- Validation: s.getDataValidationHelper(), createExplicitListConstraint (list max 255 chars), s.addValidationData
- Conditional formatting: s.getSheetConditionalFormatting(), rules + fills.
- Tables / names: XSSFSheet.createTable(area), wb.createName() named ranges.
- Pivot: createPivotTable(area, topLeft), row/column labels, addColumnLabel
- Charts: XSSFDrawing.createChart(anchor) (XDDF), images via addPicture

9. SXSSF (streaming write)

```
try (SXSSFWorkbook wb = new SXSSFWorkbook(100); // window
    OutputStream out = ...) {
    XSSFSheet s = wb.createSheet("Loans");
    s.trackAllColumnsForAutoSizing(); // before rows
    ... s.createRow(i) ... s.flushRows(50); // optional
    wb.write(out);
} // close() deletes temp files (5.3+); dispose() older
```

Only the last window rows stay in memory; rows already flushed cannot be read or edited. Keep one shared style set; one thread per workbook.

10. Streaming read: XSSFWorkbook + SheetContentsHandler

```
try (OPCPackage pkg =
    OPCPackage.open(file, PackageAccess.READ)) {
    XSSFWorkbook rd = new XSSFWorkbook(pkg);
    var sst = new ReadOnlySharedStringsTable(pkg);
    XMLReader xr = XMLHelper.newXMLReader();
    xr.setContentHandler(new XSSFSheetXMLHandler(
        rd.getStylesTable(), sst, handler,
        new DataFormatter(), false));
    try (InputStream in = rd.getSheetsData().next()) {
        xr.parse(new InputSource(in));
    } // handler: startRow, cell(ref,text,comment), endRow
```

11. XWPF skeleton

```
try (XWPFDocument d = new XWPFDocument()) {
    XWPFParagraph p = d.createParagraph();
    p.setStyle("Heading1");
    XWPFRun r = p.createRun(); r.setText("Hi"); r.setBold(true);
    XWPFTable t = d.createTable(2, 3);
    r.addPicture(in, Document.PICTURE_TYPE_PNG, "a.png",
        Units.toEMU(200), Units.toEMU(100));
    d.createHeader(HeaderFooterType.DEFAULT)
        .createParagraph().createRun().setText("Bookshelf");
    d.write(out);
}
```

Footer page number and TOC are field codes (PAGE, TOC) added via CT objects; Word refreshes the TOC with enforceUpdateFields().

12. Placeholder replacement rule

A placeholder such as \${title} may be split across several runs. Search the joined text of the paragraph's runs, then write the replacement into the first run touched and clear the others. Also visit table cells, headers and footers. Replace with real values only; never evaluate user text.

HWPFF: legacy .doc (poi-scratchpad): HWPFDocument, WordExtractor, read mostly; write via templates only.

13. Text extraction and Tika

```
try (var ex = ExtractorFactory.createExtractor(in)) {
    String text = ex.getText(); // .xlsx .docx .xls .doc
}
String t = new Tika().parseToString(in); // detect + parse
// Tika: AutoDetectParser + BodyContentHandler(maxChars)
```

Properties: wb.getProperties().getCoreProperties().setTitle(..), getExtendedProperties(), getCustomProperties().addProperty(..).

14. Encryption (Agile)

```
EncryptionInfo info =
    new EncryptionInfo(EncryptionMode.agile);
Encryptor enc = info.getEncryptor();
enc.confirmPassword(pwd); // pwd from configuration
try (POIFSFileSystem fs = new POIFSFileSystem()) {
    try (OutputStream os = enc.getDataStream(fs)) {
        wb.write(os);
    } // close the stream first
    fs.writeFileSystem(out);
} // read: WorkbookFactory.create(file, pwd)
```

Sheet/workbook protection is not encryption.

15. Security limits and formula injection

- ZipSecureFile.setMinInflateRatio(0.01); // zip bombs
- ZipSecureFile.setMaxEntrySize(100L << 20);
- ZipSecureFile.setMaxTextSize(10L << 20);
- ZipSecureFile.setMaxFileCount(1000);
- IOUTils.setByteArrayMaxOverride(50_000_000);
- Untrusted input: catch Throwable, cap memory/time, parse in a separate process or sandbox, secure temp dir.
- DOCTYPE / XXE refused since 5.1; use a recent 5.x (CVE-2025-31672 fixed in 5.4.0).
- Formula injection: user text goes in with setCellValue(String) only; never setCellFormula(userInput). In CSV, prefix cells that start with = + - @ tab or CR.

16. Spring download and upload

```
@GetMapping("/books.xlsx")
ResponseBody<StreamingResponseBody> dl() {
    StreamingResponseBody b = out -> svc.write(out);
    return ResponseEntity.ok()
        .header(HttpHeaders.CONTENT_DISPOSITION,
            ContentDisposition.attachment()
                .filename("books.xlsx", UTF_8).build().toString())
        .contentType(MediaType.parseMediaType(XLSX))
        .body(b);
}
@PostMapping(consumes = MULTIPART_FORM_DATA_VALUE)
void up(@RequestParam MultipartFile file) { ... }
// spring.servlet.multipart.max-file-size=10MB
// spring.servlet.multipart.max-request-size=10MB
```

Type	MIME type
.xlsx	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
.docx	application/vnd.openxmlformats-officedocument.wordprocessingml.document
.xls / .doc	application/vnd.ms-excel / application/msword

17. Import validation pattern

1 size cap (multipart limit) > 2 extension allow-list > 3 magic bytes with FileMagic.valueOf(in) (buffered stream), reject macro-enabled .xslm/.docm > 4 open under the ZipSecureFile limits, in a sandbox > 5 map the header row to columns by name > 6 validate each row, collect (row, column, message) > 7 return an annotated error workbook > 8 persist valid rows in one transaction. Large files: SAX reader + Spring Batch (spring-batch-excel 0.3.0).

18. Limits

	.xlsx	.xls
Rows / sheet	1,048,576	65,536
Columns / sheet	16,384 (XFD)	256 (IV)
Chars / cell	32,767	32,767
Cell styles	64,000	4,000
Fonts	-	32,767
Sheet name	31 chars	31 chars
List validation	255 chars	255 chars

19. 5.x to 6.0 changes

Version	Change
4.0	CellType enum replaces int constants; SemVer
5.0	poi-ooxml-lite / -full jars; JPMS names
5.1	log4j-api logging; DOCTYPE refused; IFS, SWITCH
5.2	XLOOKUP, XMATCH; DataFormatter cached-value option
5.3	SXSSF removes temp files on close
5.4	stricter zip parsing; duplicate entries rejected
5.5	XLSB read API (current: 5.5.1)
6.0	not released: JDK 17, removed deprecated APIs, DataFormatter cached values

Baseline 2026-10-09. Docs: https://albertoirurueta.github.io/docs, Office Files section.